

 Home  Java  Programs  OOPs  String  Exception  Multithreading

 [SCROLL TO TOP](#)

Types of Garbage Collector in Java

In Java, **garbage collection** is a mechanism that provides automatic memory management. It is done by the JVM. It need not to handle object allocation and deallocation by the programmer. In the previous sections, we have also discussed how garbage collection works.

If you are not introduced to garbage collection, we suggest that first, go through the garbage collection, **Java heap**, and **memory management**. In this section, we will discuss the **types of garbage collection in Java**.

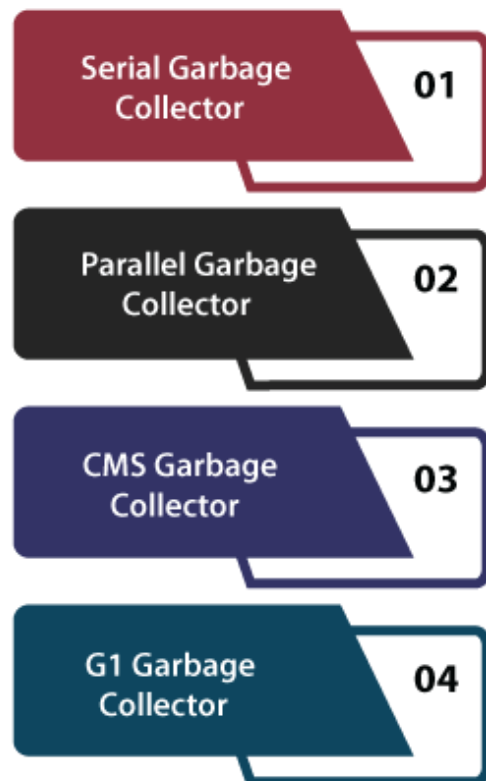
Types of Garbage Collector

There are **four** types of the garbage collector in **Java** that can be used according to the requirement:

- Serial Garbage Collector
- Parallel Garbage Collector
- Concurrent Mark Sweep (CMS) Garbage Collector
- Garbage First (G1) Garbage Collector

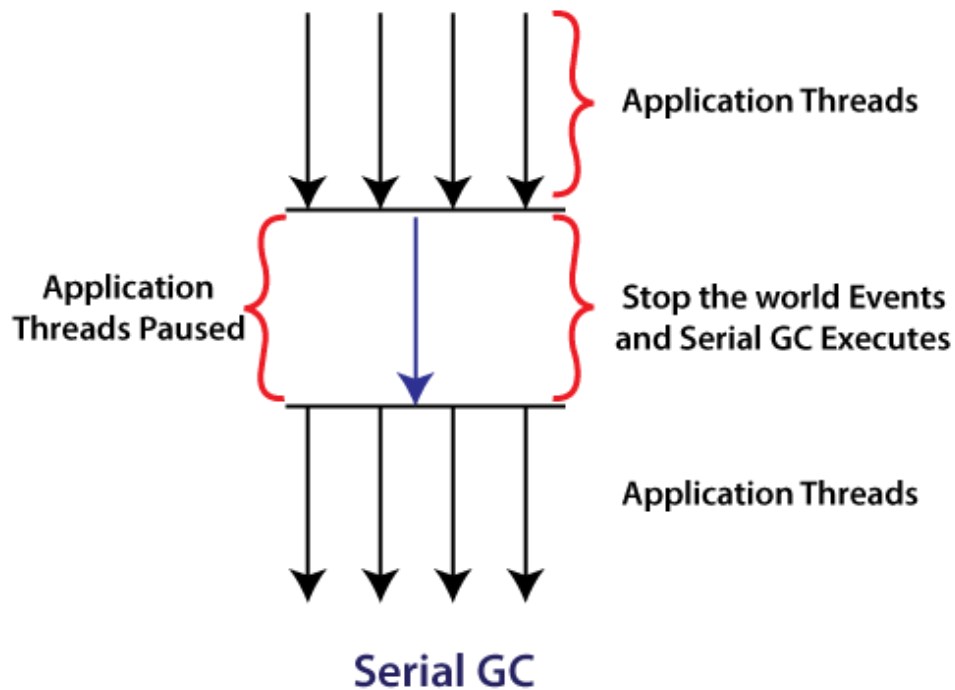
The performance and working of each garbage collector are completely different. It has its own pros and cons. Java allows us to choose any one garbage collector that is to be used by the **JVM**. For the selection of GC, we need to pass JVM arguments. It is difficult to select the right garbage collector for the application. Let's discuss each garbage collector one by one.

Types of Garbage Collector in Java



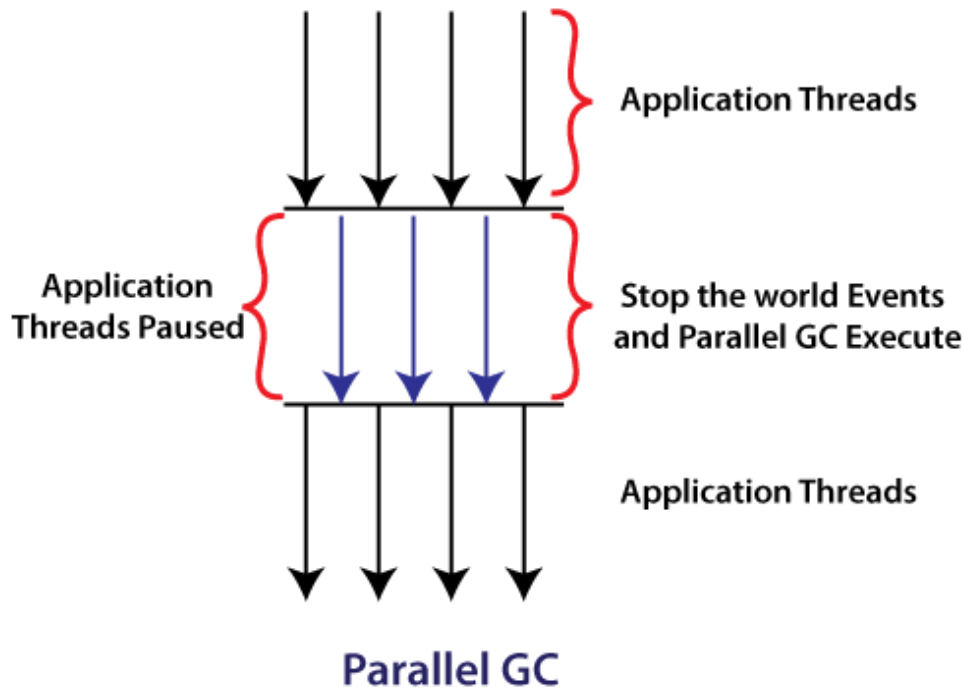
Serial Garbage Collector

Serial Garbage collector is well-matched for **single-threaded environments**. It uses the **only** thread for garbage collection. It works by holding all the threads of an application. It means that threads of the application **freeze** by the serial garbage collector during the garbage collection process and the process is known as **stop the world event**. Avoid the use of serial GC in the server environment. We can use it for simple programs. If you want to use the serial garbage collector, execute the - **XX:+UseSerialGC** JVM argument to activate it.



Parallel Garbage Collector

Parallel Garbage Collector is the **default GC** used by the JVM. The working of the parallel garbage collector is the same as the serial garbage collector. The only difference between serial and parallel garbage collector is that serial garbage collector uses a **single** thread for garbage collection process while the parallel garbage collector uses **multiple** threads for the garbage collection. Parallel GC can use multiple CPUs to speed up the application throughput. So, it is also known as **throughput collector**. It is used if we want to execute a long process (like batch processing) and where long pauses are acceptable. If you want to use the parallel garbage collector, execute the - **XX:+UseParallelGC** JVM argument to activate it.



We can also use the following JVM arguments in parallel GC:

JVM Argument	Description
-XX:ParallelGCThreads=<n>	It controls the number of GC threads (n).
-XX:MaxGCPauseMillis=<t>	It specifies the maximum pause time*.
-XX:GCTimeRatio=<n>	It specifies the maximum throughput target**.

***Pause Time:** The gap between two GC.

****Throughput Target:** The time spent during the garbage collection versus the time spent outside of garbage collection is called throughput target.

Concurrent Mark and Sweep (CMS) Garbage Collector

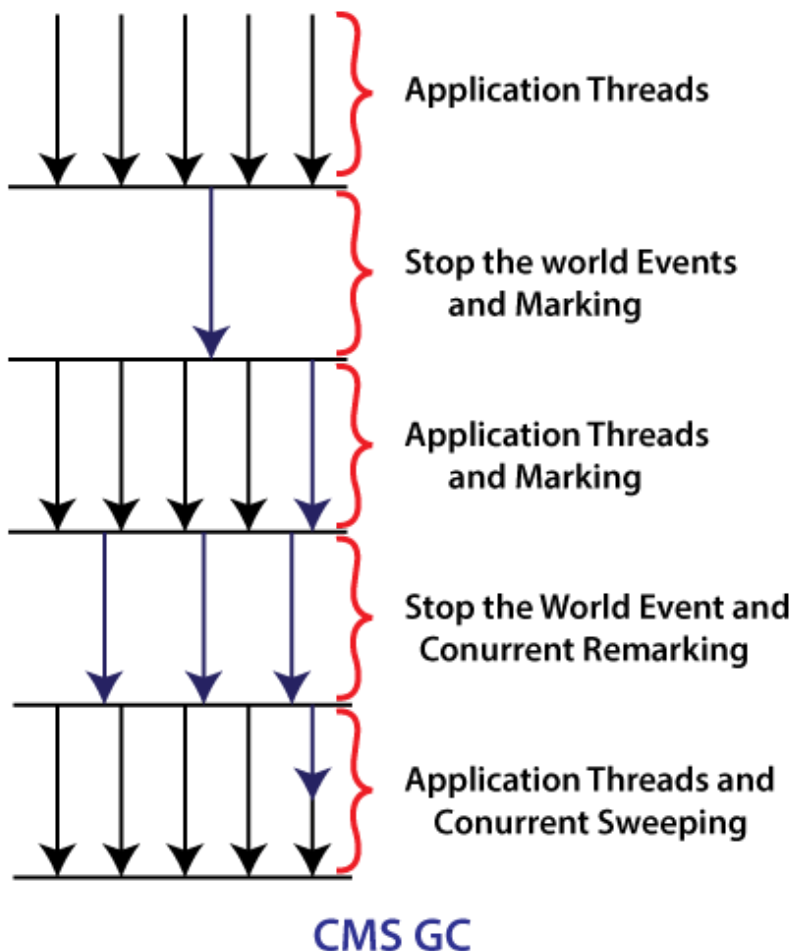
CMS uses multiple threads that scan the heap and during the scanning, it marks the instances for eviction, after scanning, it sweeps the marked instances. It does not freeze the application's threads during the garbage collection. GC threads concurrently execute with the application's threads. For this reason, it uses more CPU in comparison to other GC. It is also known as the **concurrent low pause collector**. It also freezes all the threads of the application only if it satisfies the following two scenarios:

- while marking the referenced objects in the tenured generation region.
- if any change is made to the heap memory in parallel during the garbage collection process.

AD



We can use multiple CPUs for better application throughput. We should use a CMS garbage collector if we have more CPUs for use. Hence, it has an advantage over the parallel garbage collector. If you want to use a CMS garbage collector, execute the **-XX:+UseParNewGC** JVM argument to activate it. We can also set the number of GC threads by using the **-XX:ParallelCMSThreads=<n>** JVM argument.



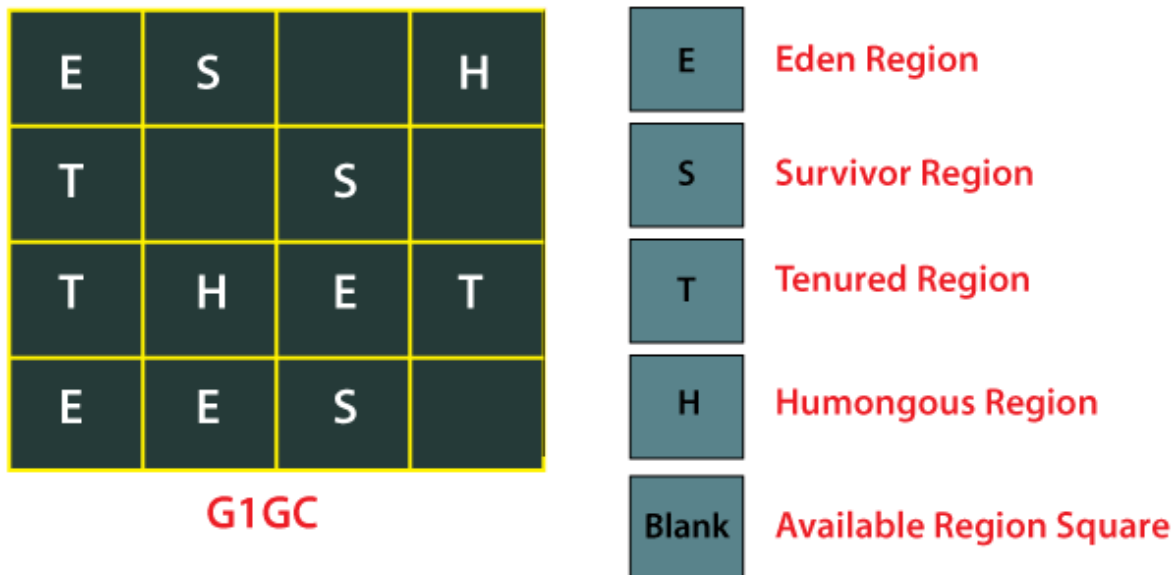
Note: The JVM argument **-XX:+UseConcMarkSweepGC** has been deprecated because a warning message is issued when it is requested on the command line.

Garbage First (G1) Garbage Collector

The G1 garbage collector is used if we have a large (more than 4GB) memory (heap space). It divides the heap into equal-sized (usually 1MB to 32MB) chunks, prioritizes them, and then performs the parallel garbage collection on that chunks based on the priority.

The Eden, survivors, and old areas use this equal-sized region for the memory allocation of the objects. Apart from these memory regions, there are two more types of regions presented in the G1 GC:

- **Humongous:** It is used if the object sized is large.
- **Available:** It represents the unoccupied space.



G1 GC shows a concurrent global marking phase to determine the live and dead objects throughout the heap. After the completion of the mark phase, G1 collects the information of regions that contains the most garbage objects. After that these regions are swept first. If you want to use the G1 garbage collector, execute the **-XX:+UseG1GC** JVM argument to activate it.

Stop the World Event

It is a situation when the garbage collector performs the garbage collection (GC) and stops all the application's threads until the GC process is not completed. The process is known as **Stop the World (STW)** events.

Improvement in Garbage Collector Since Java 8

In Java 8, the G1 garbage collector has been updated. The updated GC provides the -**XX:+UseStringDeduplication** JVM argument that optimizes the heap memory. It removes the duplicate String values to a single char[] array.

AD



Garbage Collection JVM Options

Java garbage collection key options are as follows:

AD



JVM Arguments

The table describes the arguments that can be used to instruct the JVM.

Option	Description
-XX:+UseSerialGC	Serial Garbage Collector
-XX:+UseParallelGC	Parallel Garbage Collector
-XX:+UseConcMarkSweepGC	CMS Garbage Collector
-XX:ParallelCMSThreads=	CMS Collector - number of threads to use
-XX:+UseG1GC	G1 Garbage Collector
Some Other Important Arguments	
-XX:InitiatingHeapOccupancyPercent=<n>	It controls the heap occupancy after starting the concurrent cycle. Here, n is the percentage of heap space. The default percentage is 45 .

-XX:G1MixedGCLiveThresholdPercent=<t>	If a live object in the OLD region exists with a value greater than or equal to this option, it is excluded from the GC object. Here, t is the time in milliseconds . The default value is 65 .
-XX:G1HeapWastePercent=<r>	It specifies how many regions are allowed to be wasted. Here, r is the number of regions. The default value is 10 .

GC Optimization Options

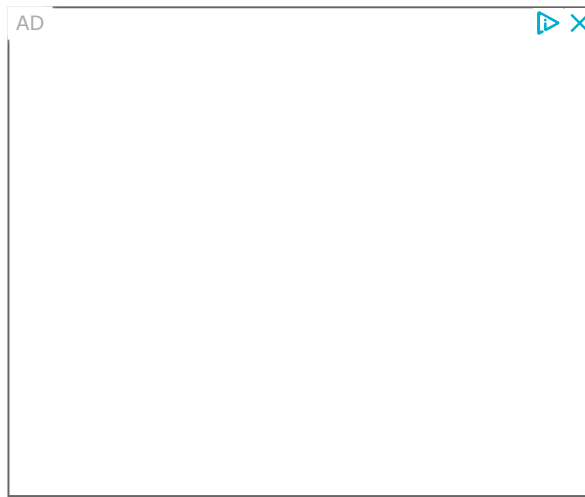
Option	Description
-Xms	It specifies the initial heap size.
-Xmx	It maximizes the heap size.
-Xmn	It is used to specify the size of the Young Generation.
-XX:PermSize	Initial Permanent Generation size
-XX:MaxPermSize	It is used to specify the maximum size of Permanent Generation (PermGen).

Usage of JVM GC Options

```
java -Xmx12m -Xms3m -Xmn1m -XX:PermSize=20m -XX:MaxPermSize=20m -XX:+UseSerialGC -jar java-application.jar
```

We have discussed all four types of Java garbage collectors. But it is difficult to select one because it depends on the application scenario, availability of hardware, and throughput requirements.

[< Prev](#)
[Next >](#)



For Videos Join Our Youtube Channel: [Join Now](#)

Feedback

- Send your Feedback to feedback@javatpoint.com

Help Others, Please Share



Learn Latest Tutorials



Splunk



SPSS



Swagger



Transact-SQL



Tumblr



ReactJS



Regex



Reinforcement
Learning



R Programming
tutorial
R Programming



RxJS tutorial
RxJS



React Native
tutorial
React Native



Python Design
Patterns
Python Design
Patterns



Python Pillow
tutorial
Python Pillow



Python Turtle
tutorial
Python Turtle



Keras tutorial
Keras

Preparation



Aptitude
Aptitude



Logical
Reasoning
Reasoning



Verbal Ability
Verbal Ability



Interview
Questions
Interview Questions



Company
Interview
Questions
Company Questions

Trending Technologies



Artificial
Intelligence
Artificial
Intelligence



AWS Tutorial
AWS



Selenium
tutorial
Selenium



Cloud
Computing
Cloud Computing



Hadoop tutorial
Hadoop



ReactJS
Tutorial



Data Science
Tutorial



Angular 7
Tutorial

	ReactJS  Blockchain Tutorial Blockchain	Data Science  Git Tutorial Git	Angular 7  Machine Learning Tutorial Machine Learning
 DevOps Tutorial DevOps			

B.Tech / MCA

 DBMS tutorial DBMS	 Data Structures tutorial Data Structures	 DAA tutorial DAA	 Operating System Operating System
 Computer Network tutorial Computer Network	 Compiler Design tutorial Compiler Design	 Computer Organization and Architecture Computer Organization	 Discrete Mathematics Tutorial Discrete Mathematics
 Ethical Hacking Ethical Hacking	 Computer Graphics Tutorial Computer Graphics	 Software Engineering Software Engineering	 html tutorial Web Technology
 Cyber Security tutorial Cyber Security	 Automata Tutorial Automata	 C Language tutorial C Programming	 C++ tutorial C++
 Java tutorial Java	 .Net Framework tutorial tutorial	 Python tutorial Python	 List of Programs Programs

.Net



Control
Systems tutorial
Control System



Data Mining
Tutorial
Data Mining



Data
Warehouse
Tutorial
Data Warehouse

AD

